



《算法设计与分析》 期末实验报告

姓名：冯怡然

学号：2411434

班级：人工智能特色班 日期：2026 年 5 月

南开大学算法·设计与分析

1 实验目的

本实验围绕无向带权图的最小生成树问题展开，要求分别利用 Kruskal 算法和 Prim 算法编写 C++ 程序，求解题目所给图对应的最小生成树。通过实验，需要理解两种贪心算法的基本思想、适用数据结构、执行过程以及结果差异，并能够结合具体图结构分析最小生成树的边集和总权值。

2 实验内容

题目要求如下：利用 Kruskal 算法和 Prim 算法，编写 C++ 程序求解图 1 对应的最小生成树，并在实验报告中说明算法原理、给出代码、展示运行结果并分析结果。

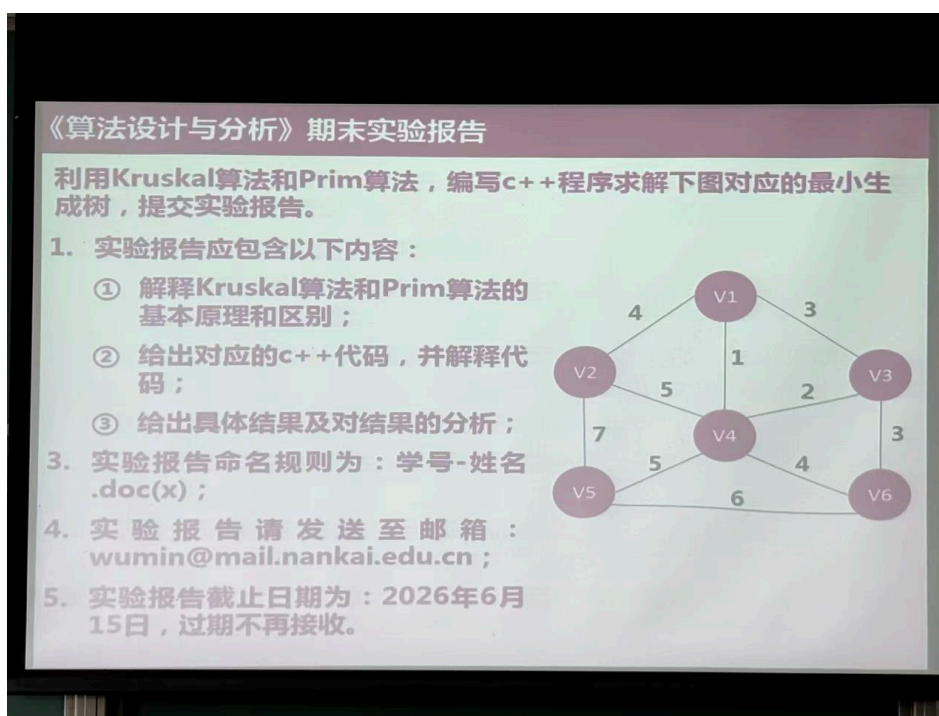


图 1: 题目给定的无向带权图

根据图中信息，可将无向边整理为表 1。

表 1: 图中的边及权值

边	权值	说明	边	权值	说明
V1-V2	4	无向边	V1-V3	3	无向边
V1-V4	1	无向边	V2-V4	5	无向边
V2-V5	7	无向边	V3-V4	2	无向边
V3-V6	3	无向边	V4-V5	5	无向边
V4-V6	4	无向边	V5-V6	6	无向边

3 算法原理

3.1 Kruskal 算法

Kruskal 算法是一种按边选择的贪心算法。它先将图中所有边按权值从小到大排序，然后依次考察每条边：如果当前边连接的两个顶点属于不同连通分量，则加入最小生成树；如果加入后会形成环，则跳过该边。为了快速判断两个顶点是否已经连通，程序通常使用并查集维护连通分量。

对于有 n 个顶点的连通无向图，最小生成树应包含 $n - 1$ 条边。本题共有 6 个顶点，因此算法在选出 5 条边后结束。Kruskal 算法更关注边的全局权值顺序，适合边数较少或边排序开销可以接受的场景。

3.2 Prim 算法

Prim 算法是一种按顶点扩展的贪心算法。算法从某一个起始顶点出发，维护已经加入最小生成树的顶点集合，每一步从所有跨越该集合与未访问顶点集合的边中选择权值最小的一条，将对应的新顶点加入生成树。重复该过程，直到所有顶点均被加入。

本实验程序中 Prim 算法从 V1 开始，并使用优先队列维护候选边。Prim 算法的核心是“从当前生成树向外扩展”，因此它的选择过程与 Kruskal 不同，但只要图连通并且每一步按最小权值边扩展，最终总权值仍为最小。

3.3 Kruskal 与 Prim 的区别

Kruskal 算法从边集出发，先排序再用并查集判断是否成环；Prim 算法从顶点出发，每次选择当前生成树到外部顶点的最小边。前者更像是在全图范围内筛选边，后者更像是逐步扩大一棵树。两种算法都基于贪心思想，但维护对象不同：Kruskal 维护连通分量，Prim 维护已访问顶点集合和候选边集合。

4 程序设计与代码实现

程序首先用结构体 `Edge` 存储边的两个端点和权值。Kruskal 算法部分先对边进行排序，再通过并查集判断是否可以加入当前生成树。Prim 算法部分先建立邻接表，再从 V1 出发，用优先队列不断取出权值最小的候选边。最后分别输出两种算法得到的边集和总权值。

```
1 #include <algorithm>
2 #include <functional>
3 #include <iomanip>
4 #include <iostream>
5 #include <queue>
6 #include <string>
7 #include <tuple>
8 #include <vector>
9 using namespace std;
10 // Edge stores one undirected weighted edge: u-v with weight w.
11 struct Edge {
12     int u;
13     int v;
14     int w;
```

```

15 };
16 // Disjoint Set Union is used by Kruskal to check whether an edge creates a cycle.
17 class DSU {
18 public:
19     explicit DSU(int n) : parent(n + 1), rank(n + 1, 0) {
20         for (int i = 1; i <= n; ++i) {
21             parent[i] = i;
22         }
23     }
24     // Find the root of x and compress the path to speed up later searches.
25     int find(int x) {
26         if (parent[x] != x) {
27             parent[x] = find(parent[x]);
28         }
29         return parent[x];
30     }
31     // Merge two sets. Return true if the edge can be added to the MST.
32     bool unite(int a, int b) {
33         int rootA = find(a);
34         int rootB = find(b);
35         if (rootA == rootB) {
36             return false;
37         }
38         if (rank[rootA] < rank[rootB]) {
39             swap(rootA, rootB);
40         }
41         parent[rootB] = rootA;
42         if (rank[rootA] == rank[rootB]) {
43             ++rank[rootA];
44         }
45         return true;
46     }
47 private:
48     vector<int> parent;
49     vector<int> rank;
50 };
51 // Convert vertex number to its name in the problem, such as 1 -> V1.
52 string vertexName(int x) {
53     return "V" + to_string(x);
54 }
55 // Print all selected edges and the total weight of the MST.
56 void printResult(const string &algorithmName, const vector<Edge> &mst, int
    totalWeight) {
57     cout << algorithmName << " result:\n";
58     cout << left << setw(12) << "Edge" << "Weight\n";
59     cout << "-----\n";
60     for (const auto &edge : mst) {
61         cout << left << setw(12) << (vertexName(edge.u) + "-" +
            vertexName(edge.v)) << edge.w << '\n';
62     }

```

```

63     cout << "Total weight: " << totalWeight << "\n\n";
64 }
65 // Kruskal algorithm:
66 // 1. Sort all edges by weight.
67 // 2. Scan edges from small to large.
68 // 3. Add an edge only when it connects two different components.
69 // 4. Stop when n - 1 edges have been selected.
70 pair<vector<Edge>, int> kruskal(int n, vector<Edge> edges) {
71     sort(edges.begin(), edges.end(), [](const Edge &a, const Edge &b) {
72         if (a.w != b.w) {
73             return a.w < b.w;
74         }
75         if (min(a.u, a.v) != min(b.u, b.v)) {
76             return min(a.u, a.v) < min(b.u, b.v);
77         }
78         return max(a.u, a.v) < max(b.u, b.v);
79     });
80     DSU dsu(n);
81     vector<Edge> mst;
82     int totalWeight = 0;
83     for (const auto &edge : edges) {
84         if (dsu.unite(edge.u, edge.v)) {
85             mst.push_back(edge);
86             totalWeight += edge.w;
87             if (static_cast<int>(mst.size()) == n - 1) {
88                 break;
89             }
90         }
91     }
92     return {mst, totalWeight};
93 }
94 // Prim algorithm:
95 // Start from one vertex, always choose the smallest edge from visited vertices
96 // to unvisited vertices, and grow the MST step by step.
97 pair<vector<Edge>, int> prim(int n, const vector<Edge> &edges, int start) {
98     vector<vector<pair<int, int>>> graph(n + 1);
99     for (const auto &edge : edges) {
100         graph[edge.u].push_back({edge.v, edge.w});
101         graph[edge.v].push_back({edge.u, edge.w});
102     }
103     // State means (weight, from, to). The priority queue works as a min-heap.
104     using State = tuple<int, int, int>;
105     priority_queue<State, vector<State>, greater<State>> pq;
106     vector<bool> visited(n + 1, false);
107     vector<Edge> mst;
108     int totalWeight = 0;
109     visited[start] = true;
110     for (size_t i = 0; i < graph[start].size(); ++i) {
111         int to = graph[start][i].first;
112         int weight = graph[start][i].second;

```

```

113     pq.push(State(weight, start, to));
114 }
115 while (!pq.empty() && static_cast<int>(mst.size()) < n - 1) {
116     State current = pq.top();
117     pq.pop();
118     int weight = get<0>(current);
119     int from = get<1>(current);
120     int to = get<2>(current);
121     if (visited[to]) {
122         continue;
123     }
124     visited[to] = true;
125     mst.push_back(Edge{from, to, weight});
126     totalWeight += weight;
127     for (size_t i = 0; i < graph[to].size(); ++i) {
128         int next = graph[to][i].first;
129         int nextWeight = graph[to][i].second;
130         if (!visited[next]) {
131             pq.push(State(nextWeight, to, next));
132         }
133     }
134 }
135 return {mst, totalWeight};
136 }
137 int main() {
138     const int n = 6;
139     // Input all undirected edges from the graph in the assignment picture.
140     vector<Edge> edges = {
141         {1, 2, 4}, {1, 3, 3}, {1, 4, 1},
142         {2, 4, 5}, {2, 5, 7},
143         {3, 4, 2}, {3, 6, 3},
144         {4, 5, 5}, {4, 6, 4},
145         {5, 6, 6}
146     };
147     pair<vector<Edge>, int> kruskalResult = kruskal(n, edges);
148     vector<Edge> kruskalMst = kruskalResult.first;
149     int kruskalWeight = kruskalResult.second;
150     pair<vector<Edge>, int> primResult = prim(n, edges, 1);
151     vector<Edge> primMst = primResult.first;
152     int primWeight = primResult.second;
153     printResult("Kruskal", kruskalMst, kruskalWeight);
154     printResult("Prim(start from V1)", primMst, primWeight);
155     return 0;
156 }

```

Listing 1: Kruskal 与 Prim 求解最小生成树的 C++ 程序

5 实验结果

```
Kruskal result:
Edge      Weight
-----
V1-V4      1
V3-V4      2
V3-V6      3
V1-V2      4
V4-V5      5
Total weight: 15

Prim(start from V1) result:
Edge      Weight
-----
V1-V4      1
V4-V3      2
V3-V6      3
V1-V2      4
V4-V5      5
Total weight: 15
```

图 2: 实验结果

6 结果分析

从运行结果可以看出，两种算法得到的最小生成树总权值均为 15，说明程序计算结果一致。由于图中存在相同权值的边，例如权值为 3 的 V1-V3 与 V3-V6，以及权值为 4 的 V1-V2 与 V4-V6，因此不同算法或不同的并列边处理顺序可能得到边集略有差异的最小生成树。但是，只要满足连通所有顶点、包含 5 条边且不形成环，并且总权值达到最小，结果就是正确的最小生成树。

本题中，V1-V4 权值为 1，是全图中权值最小的边，必然会被优先考虑。V3-V4 权值为 2，也能够以较低代价连接 V3。随后 V3-V6 权值为 3 将 V6 接入生成树，V1-V2 权值为 4 将 V2 接入生成树，最后 V4-V5 权值为 5 将 V5 接入。此时 6 个顶点全部连通，共选取 5 条边，构成一棵生成树，总权值为

$$1 + 2 + 3 + 4 + 5 = 15.$$

Kruskal 算法在排序后的全局边序列中跳过了会产生回路的边，例如 V1-V3 在本程序中会被跳过，因为此时 V1 与 V3 已经通过 V1-V4 和 V3-V4 连通。Prim 算法则只从当前已形成的树向外选择最短边，因此它的边选择顺序更加依赖起始点。本实验从 V1 出发，能够直观展示 Prim 算法逐步扩展生成树的过程。

7 实验总结

通过本次实验，完成了 Kruskal 算法和 Prim 算法求解最小生成树的 C++ 实现，并对题目给定图进行了求解。实验表明，Kruskal 和 Prim 虽然选边角度不同，但都遵循贪心策略，并能在连通无向带权图上得到最小生成树。通过对比两种算法，可以进一步理解并查集、优先队列和图的邻接表表示在图算法实现中的作用。